



Alexandru - Mihai Barbu

✉ Email: alexbrb27@gmail.com ☎ Phone: (+40) 0748923851

🌐 Website: <https://github.com/AlexBrb278>

🌐 Website: <https://linkedin.com/in/alex-barbu-5ba183264>

📅 Date of birth: 27/08/2002 🇷🇴 Nationality: Romanian

ABOUT ME

I'm a mechatronics engineer having finished the Faculty of Mechanical and Mechatronics. Robotics drew me in from the start, but (jumping on the bandwagon) I am fascinated by AI and the quick growth it had over the last few years. I am curious about the next big step in robotics which would be about robots that actually perceive the world and how that would work. That's why my dissertation is about human-robot interaction and why I am seriously considering a PhD to dig deeper into this topic.

SKILLS

Programming Skills

Python | JavaScript | HTML | Git | MATLAB | ROS, ROS2 | C/C++

AI Skills

PyTorch | TensorFlow | Artificial Intelligence frameworks: Caffe, Darknet, YOLO

3D Design

Fusion-360 | SolidEdge 3D | Knowledge of Altium Designer, Proteus, SOLIDWORKS, AutoCAD, Multisim and Node-RED Software tools

Networking Skills

Communication protocols (MODBUS MQTT CAN PROFIBUS) | TCP IP | UDP | Radio Communication

Languages

English (Fluent) | Romanian (native)

EDUCATION AND TRAINING

[2025 – Current]

Master's Degree in Robotics and Automation

Faculty of Automatic Control and Computer Science

City: Bucharest | Country: Romania | Field(s) of study: Robotics and Automation | Level in EQF: EQF level 7

Customized curriculum drawing half the courses from the Artificial Intelligence Master program, covering Computer Vision, NLP, and ML theory to complement the robotics core with the areas of AI I find most compelling.

[2021 – 2025]

Mechatronics engineer

UPB, Faculty of Mechanical Engineering and Mechatronics

Field(s) of study: Mechatronics and Robotics | Final grade: 9.50 | NQF Level: 6 | Thesis: Delta Robot with Computer Vision for Object Sorting

- Designed, 3D-printed, and assembled a fully functional delta robot capable of sorting small objects by type using an electromagnet.
- Trained a custom YOLOv8 model on self-collected images of nuts and washers for accurate object detection.
- Implemented a complete system on Raspberry Pi 5 — real-time detection, position calculation, inverse kinematics, and stepper motor control.
- Developed path planning and collision avoidance algorithms to optimize the sorting cycle.
- Ran simulations in MATLAB and Fusion360 to verify motor specs, trajectory performance, and structural load tolerance.

WORK EXPERIENCE

MarcTel S.I.T.

[11/2025 – Current]

Software Engineer

Robotic Arm Integration on Drone Architected full-stack ground control system for drone-mounted 6-DOF robotic arm. Developed ROS2 control software (Jetson Orin NX) managing concurrent state machine (flight modes, waypoint navigation, arm control, gripper feedback) with

smooth trajectory interpolation and per-axis collision limits. Implemented adaptive radio telemetry service monitoring RSSI with dynamic transmit power maintaining signal quality. Designed PyQt5 ground control station enabling MQTT-based arm commands from laptop through radio. Added Arduino MKR input device with dual joysticks and programmable buttons (analog deadzone filtering). Integrated multi-protocol stack: MQTT (commands), MAVLink (flight control), UDP (Jetson-Pixhawk), RTSP (video streaming), serial (input device). Tech: ROS2, Jetson Orin NX, Dynamixel, Pixhawk 6, MQTT, PyQt5, Arduino, MAVLink, RTSP.

C2 Ground Station & UAV File Relay System Architected end-to-end data relay pipeline for autonomous underwater vehicle (AUV) sonar/bathymetry file transfer to ground control station through an UAV relay. Designed and implemented cross-platform TCP file transfer protocol (C++17, Winsock/POSIX) with validated wire framing (filename + size header followed by 64 KB chunked streaming). Implemented three-tier transfer stack: sender (ARM64 Linux SBC on AUV), relay node (UAV SBC receiving and retransmitting), receiver (Windows C2 workstation), with symmetric progress telemetry (transfer speed, ETA) reported every second. Designed Qt6 ground station GUI from scratch with reactive data flow architecture and offline-first interactive mapping.

- Eliminated UI blocking: QObject worker + QThread pattern marshaling network I/O to background thread; progress signals delivered to main thread via Qt queued connections driving reactive UI updates
- Reactive state architecture: Central MissionModel hub—workers write exclusively via slots, widgets read exclusively via signals; no direct hardware-to-UI coupling
- Real-time visualization: Animated AUV→UAV→C2 pipeline diagram with live progress bars, speed readout, ETA calculation synchronized to TCP receive loop
- Offline-first mapping: MapLibre-GL (QWebEngineView) backed by custom Qt TCP tile server (MBTiles/SQLite, thread-pooled); dynamic layers only requested after connectivity confirmed. Real-time AUV and UAV position tracking on live map during active relay operations

Vision Technology Development

[06/2024 – 10/2025] **IoT Engineer**

Worked on multiple projects, with the one I am most proud of:

Smart Parking System: Architected distributed embedded vision system for intelligent parking monitoring. Developed high-performance C backend using Axis SDK and quantized YOLOv5 model optimized for ARTPEC-8 DLPU; containerized via Docker into .eap package for direct camera deployment. Implemented multi-threaded inference pipeline with MQTT client for asynchronous image publishing. Built interactive Canvas-based web UI (HTML5/JavaScript) enabling users to draw parking space ROIs; engineered spatial collision detection algorithm comparing real-time detection bounding boxes with user-defined regions. On intersection, triggered image capture and MQTT publication to downstream Python LPR service. Integrated OCR pipeline consuming MQTT stream and persisting license plates + timestamps + photos to SQLite database. Designed RESTful API (C backend) enabling 300ms-interval frontend polling with <100ms response times, achieving <500ms E2E latency.

Tech: C, Axis SDK, Docker, YOLOv5, MQTT, HTML5, JavaScript, CSS, LPR model, SQLite.

- Implemented HTTP server within C application using Axis SDK HTTP utilities, enabling frontend-backend communication.
- Designed and implemented core API endpoints:
 - POST /spaces - Accepts user-drawn ROI coordinates (normalized 1000-unit scale) and parking space identifiers; persists configuration server-side
 - POST /settings - Updates MQTT broker settings, confidence thresholds, and detection parameters with real-time backend effect
 - GET /app - Application metadata: serves device specifications, model properties, manifest configuration (friendlyName, version, vendor), MQTT broker settings, and system capabilities for frontend initialization
 - GET /status - Real-time detection retrieval: polls inference results including normalized bounding box coordinates, model inference latency (ms), confidence scores, active event states, and MQTT connection status. Optimized for 300ms polling interval with <100ms response time
 - GET /detections - Historical detection log: retrieves timestamped array of processed detections with metadata (coordinates, confidence, detection_id) for UI rendering and analytics
- Implemented HTTP status code handling (200/400/500) with coordinate bounds validation for 1000-unit normalized scale

PROJECTS

[07/2026 – Current]

Shade-aware walking navigation

- Built a geospatial pipeline that derives building heights from OpenStreetMap, projects shadow polygons using solar position math (Shapely, pysolar), and precomputes them in 30-minute time slices
- Implemented custom-weighted routing over the OSM walking graph (osmnx/networkx), scoring every street segment by sun exposure and returning fastest-vs-shadiest route comparisons via a FastAPI backend
- Shipped a SvelteKit + MapLibre mobile frontend wrapped with Capacitor for iOS/Android

[02/2026 – 05/2026]

Noisy Intent Detection for Chatbots

- Investigated robustness of intent classifiers under 7 noise types including a real ASR pipeline built with edge-tts and OpenAI Whisper.
- Applied NT-Xent contrastive fine-tuning to BERT, recovering 31 percentage points of keyboard noise accuracy with no architectural changes.
- Combined CANINE token-free encoding, contrastive learning, and prototypical classification to achieve 83% keyboard accuracy and 40% OOS recall.

[12/2023 – 05/2024]

Plant Disease Detection using Deep Learning in MATLAB

- Built a GUI that identifies diseases in tomatoes, potatoes, and bell peppers with diagnostic information and treatment recommendations.
- Trained a neural network on 9000 high-resolution plant images using Microsoft Azure for cloud compute.
- Integrated an ESP32-CAM stream into the interface, mounted on a custom mini-rover for field image capture.
- Won 1st place at “Zilele Educatiei Mecatronice 2024” in the Student Achievements in MATLAB Applications category

COURSES WITH CERTIFICATION

.

Oct 2025 - Dec 2025 **CCNAv7: Enterprise Networking, Security and Automation**

Dec 2024 - Jan 2025 **Supervised Machine Learning: Regression and Classification**

Dec 2024 - Jan 2025 **Interfacing with the Arduino/Raspberry Pi**

Oct 2024 - Jan 2025 **Python101: Modules, OOP, Web-Scraping**

Mar 2024 - Jun 2024 **CCNAv7: Switching, Routing, and Wireless Essentials**

Nov 2023 - Feb 2024 **CCNAv7: Introduction to Networks**

Oct 2023 - Jan 2024 **Solid Edge Associate Level Certification**

HONOURS AND AWARDS

[12/2025]

1st Place — CodeQuest Hackathon, Artificial Intelligence Category Awarding institution: Serviciul de Informații Externe

Built a real-time facial recognition pipeline capable of identifying individuals from a database at up to 12 meters, with or without masks. Fine-tuned YOLOv8n on a custom augmented dataset for masked/unmasked detection; benchmarked SAHI for improved recall; implemented a lightweight SVM classifier that reduced inference latency by ~ 90% compared to ArcFace and FaceNet512.

[05/2024]

1st Place — Zilele Educatiei Mecatronice 2024 Awarding institution: UPB

Student Achievements in the Field of Mechatronics at MATLAB Applications. Presented the plant disease detection system deployed on a field rover.